Bilkent University

Department of Electrical and Electronics Engineering

EE-102: Introduction to Digital Circuit Design

Lab 5 Report: Multiplexed 7-Segment Display Controller

Author: Can Kurc

ID: 22502100

Date: March 26, 2026

1 Introduction and Objective

The purpose of this lab is to design, simulate and implement a multiplexed 4-digit 7-segment display controller on the Basys3 FPGA. Since the 4 digits on the Basys3 share the same cathode pins (A-G), they cannot be driven with four different numbers simultaneously. To solve this, a time-multiplexing technique is used. By rapidly cycling through each digit one at a time, the human eye perceives all four digits as being illuminated simultaneously due to the "persistence of vision" effect.

2 Theoretical Background and Clock Division (Questions)

Before implementing the design, several hardware constraints regarding the FPGA's internal clock had to be addressed:

- **Internal Clock Frequency:** The internal clock frequency of the Basys 3 board is 100 MHz.
- **Creating a Slower Clock:** A 100 MHz clock is too fast to directly drive the display multiplexer without causing dimming or timing issues. A slower clock signal can be created by implementing a binary counter that increments on every rising edge of the 100 MHz clock. By tapping into the higher-order bits of this counter, we obtain a signal that toggles at a fraction of the original speed.
- **Arbitrary Frequencies:** It is not possible to create any arbitrary frequency using a simple binary counter. A standard binary counter divides the base frequency by powers of 2 (e.g., 100 MHz/2^N). To achieve an exact arbitrary frequency, a more complex modulus counter (which resets at a specific target integer) would be required.
- **Persistence of Vision:** For this design, a 20-bit counter was utilized. The multiplexer is driven by the top two bits (bits 19 and 18). Bit 18 flips every 262,144 clock cycles (2^{18}), resulting in a refresh frequency of approximately 381.4 Hz ($100,000,000/262,144$). This is well above the $\tilde{6}0$ Hz threshold required for the human eye, ensuring a flicker-free display without the need for complex modulus logic.

3 Design Architecture

The system was designed using a modular approach in VHDL, consisting of four primary sub-modules wired together using Vivado's IP Integrator (Block Design).

3.1 Clock Divider (clock_divider.vhd)

This module takes the 100 MHz clk as an input and drives a 20-bit internal counter. It outputs a 2-bit signal (digit_select) extracted from the 19th and 18th bits of the counter. This 2-bit signal continuously cycles through "00", "01", "10", and "11" at a rate of $\tilde{3}81$ Hz, acting as the synchronization signal for the entire system.

Listing 1: clock_divider.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity clock_divider is
6      Port ( clk : in STD_LOGIC;
7             digit_select : out STD_LOGIC_VECTOR (1 downto 0));
8  end clock_divider;
9
10 architecture Behavioral of clock_divider is
```

```
    -- Our 20-bit counter
    -- It can count from 0 up to 1,048,575
    signal refresh_counter : STD_LOGIC_VECTOR (19 downto 0) := (others => '0');
begin
    process(clk)
    begin
        if rising_edge(clk) then
            refresh_counter <= refresh_counter + 1;
        end if;
    end process;

    -- ignore the bottom 18 bits (which are flashing millions of times a second).
    -- look at the top 2 bits (bits 19 and 18).
    -- Bit 18 flips  every 262,144 clock ticks (100MHz / 262144 = 381.4 Hz).
    -- This gives a 381 Hz refresh rate

    digit_select <= refresh_counter(19 downto 18);
end Behavioral;
```

3.2 4-to-1 Multiplexer (mux_4to1.vhd)

This module routes the data from the 16 physical slide switches (sw[15:0]) to the display decoder. It uses the 2-bit digit_select signal to determine which group of 4 switches to pass forward. For example, when digit_select is "00", it passes sw[3:0]; when "01", it passes sw[7:4], and so on.

Listing 2: mux_4to1.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_4to1 is
    Port (
        sw           : in  STD_LOGIC_VECTOR (15 downto 0); -- All 16 switches
        digit_select : in  STD_LOGIC_VECTOR (1 downto 0);  -- The 2-bit clock signal
        digit_out    : out STD_LOGIC_VECTOR (3 downto 0)   -- The chosen 4 switches
    );
end mux_4to1;

architecture Behavioral of mux_4to1 is
begin
    process(digit_select, sw)
    begin
        case digit_select is
            when "00" => digit_out <= sw(3 downto 0);   -- Group 1
            when "01" => digit_out <= sw(7 downto 4);   -- Group 2
            when "10" => digit_out <= sw(11 downto 8);  -- Group 3
            when "11" => digit_out <= sw(15 downto 12); -- Group 4
            when others => digit_out <= "0000";         -- Failsafe
        end case;
    end process;
end Behavioral;
```

3.3 Anode Decoder (anode_decoder.vhd)

The Anode Decoder controls which of the four 7-segment screens is currently powered. The displays on the Basys 3 are active-low. Based on the digit_select signal, it outputs a 4-bit active-low pattern to the an ports (e.g., "1110" activates the rightmost digit, "1101" activates the second digit).

Listing 3: anode_decoder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity anode_decoder is
    Port (
        digit_select : in  STD_LOGIC_VECTOR (1 downto 0); -- The 2-bit clock signal
        an           : out STD_LOGIC_VECTOR (3 downto 0)  -- The 4 screen power wires
    );
end anode_decoder;

architecture Behavioral of anode_decoder is
begin
    process(digit_select)
    begin
        case digit_select is
            when "00" => an <= "1110"; -- Turns ON Digit 0 (Rightmost)
            when "01" => an <= "1101"; -- Turns ON Digit 1
            when "10" => an <= "1011"; -- Turns ON Digit 2
            when "11" => an <= "0111"; -- Turns ON Digit 3 (Leftmost)
            when others => an <= "1111"; -- Keeps all OFF as a failsafe
        end case;
    end process;
end Behavioral;
```

3.4 BCD to 7-Segment Decoder (binary_input_to_7_segment_display.vhd)

This module takes the 4-bit output from the multiplexer and decodes it into a 7-bit active-low signal (seg[6:0]) that drives the individual LED segments (A through G) to form hexadecimal characters (0-9, A-F).

Listing 4: binary_input_to_7_segment_display.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity binary_input_to_7_segment_display is
    Port ( bcd_in : in STD_LOGIC_VECTOR (3 downto 0);
           seg_out : out STD_LOGIC_VECTOR (6 downto 0));
end binary_input_to_7_segment_display;

architecture Behavioral of binary_input_to_7_segment_display is
begin
process(bcd_in)
    begin
        case bcd_in is
            when "0000" => seg_out <= "1000000"; -- '0'
            when "0001" => seg_out <= "1111001"; -- '1'
            when "0010" => seg_out <= "0100100"; -- '2'
            when "0011" => seg_out <= "0110000"; -- '3'
            when "0100" => seg_out <= "0011001"; -- '4'
            when "0101" => seg_out <= "0010010"; -- '5'
            when "0110" => seg_out <= "0000010"; -- '6'
            when "0111" => seg_out <= "1111000"; -- '7'
            when "1000" => seg_out <= "0000000"; -- '8'
            when "1001" => seg_out <= "0010000"; -- '9'

            -- Hex letters for inputs 10 to 15
            when "1010" => seg_out <= "0001000"; -- 'A'
            when "1011" => seg_out <= "0000011"; -- 'b'
            when "1100" => seg_out <= "1000110"; -- 'C'
            when "1101" => seg_out <= "0100001"; -- 'd'
```

```
            when "1110" => seg_out <= "0000110"; -- 'E'
            when "1111" => seg_out <= "0001110"; -- 'F'

            when others => seg_out <= "1111111"; -- Keep all off as a failsafe
        end case;
    end process;
end Behavioral;
```

3.5 Top-Level Block Design

The individual components were instantiated into a top-level block design, creating a unified RTL wrapper. The clk and sw signals act as external inputs, while the seg and an signals act as external outputs.

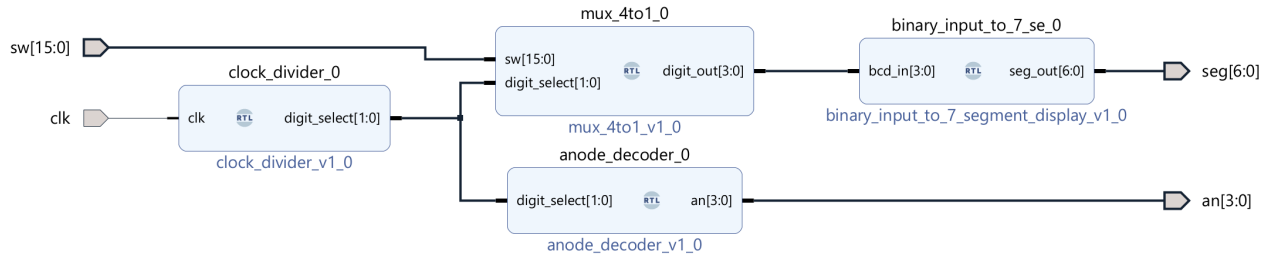


Figure 1: Vivado IP Integrator Top-Level Block Design (SevenSeg.bd)

4 Simulation Results

To verify the logic before physical implementation, a testbench (tb_SevenSeg.vhd) was created. The testbench simulated a 100 MHz clock and applied specific 16-bit values to the switches (e.g., x"4321" and x"BEEF").

Because the refresh rate was designed to be realistic ($\tilde{2}.6$ ms per digit transition), the simulation was run for 15 milliseconds to observe a full multiplexing cycle. The resulting waveform demonstrated that the an signal correctly shifted the active-low zero across all four digits, while the seg signal output the corresponding character codes in perfect synchronization.

Testbench:

Listing 5: tb_SevenSeg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_SevenSeg is
-- Testbench entities are empty
end tb_SevenSeg;

architecture Behavioral of tb_SevenSeg is
    -- 1. Declare the Vivado-generated wrapper as our component
    component SevenSeg_wrapper is
        Port (
            clk : in STD_LOGIC;
            sw  : in STD_LOGIC_VECTOR ( 15 downto 0 );
            an  : out STD_LOGIC_VECTOR ( 3 downto 0 );
            seg : out STD_LOGIC_VECTOR ( 6 downto 0 )
        );
    end component;

    -- 2. Virtual wires for simulation
```

```
    signal clk : STD_LOGIC := '0';
    signal sw  : STD_LOGIC_VECTOR ( 15 downto 0 ) := (others => '0');
    signal an  : STD_LOGIC_VECTOR ( 3 downto 0 );
    signal seg : STD_LOGIC_VECTOR ( 6 downto 0 );
    constant clk_period : time := 10 ns;

begin
    -- 3. Connect the virtual wires to your Block Design
    UUT: SevenSeg_wrapper port map (
        clk => clk,
        sw  => sw,
        an  => an,
        seg => seg
    );

    -- 4. Generate the 100 MHz clock
    clk_process :process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

    -- 5. Provide inputs to the switches
    stimulus: process
    begin
        -- Test Case 1: Display "4321"
        sw <= x"4321";

        -- Wait 15 ms so the counter has time to cycle all 4 digits
        wait for 15 ms;

        -- Test Case 2: Display "bEEF"
        sw <= x"BEEF";
        wait for 15 ms;

        wait;
    end process;
end Behavioral;
```

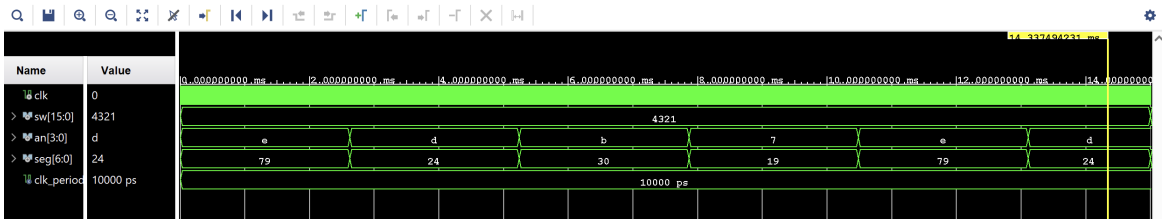


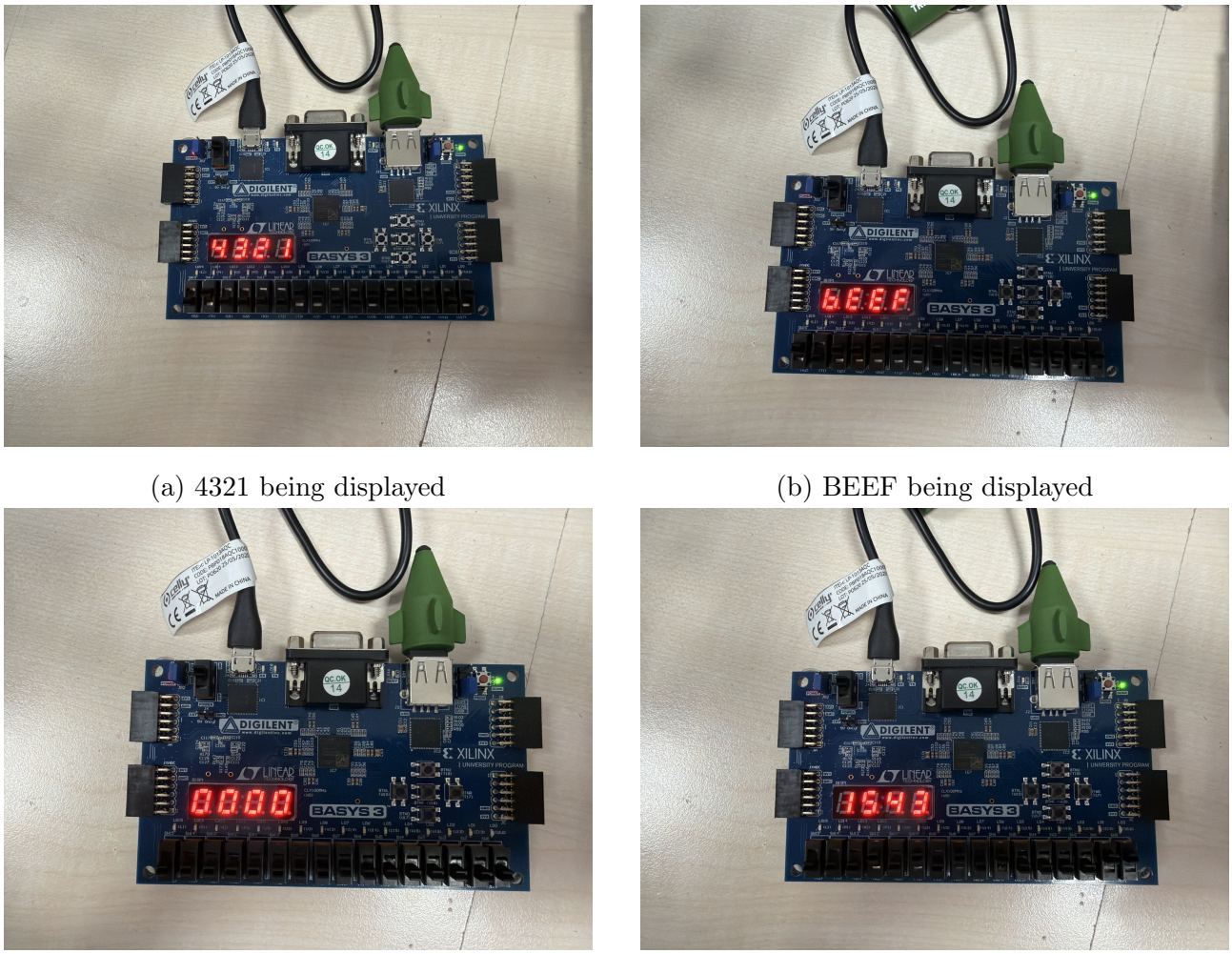
Figure 2: 15ms Vivado Behavioral Simulation Waveform

5 Hardware Implementation

An XDC constraints file was created to map the virtual ports to the physical pins on the Basys 3. Furthermore, a timing constraint (create_clock -add -name sys_clk_pin -period 10.00) was added to properly define the 100 MHz clock for Vivado's Timing Analyzer, resolving the initial TIMING-17: Non-clocked sequential cell critical warning.

The generated bitstream was successfully programmed onto the Basys 3 board. Physical testing confirmed that the display accurately output the hexadecimal values corresponding to the 16 slide

switches. The 381 Hz refresh rate proved successful, as all four digits appeared completely solid and bright with no visible flickering.



(a) 4321 being displayed

(b) BEEF being displayed

(c) 0000 being displayed

(d) 1543 being displayed

Figure 3: Hardware Verification on the Basys 3 Board

6 Conclusion

In this lab, a multiplexed 7-segment display controller was designed and deployed. By leveraging a high-speed counter to divide the FPGA's internal clock, a synchronized multiplexing system was created that bypasses the hardware limitation of shared cathode pins. The final implementation achieved the persistence of vision effect and accurately displayed 16-bit inputs in hexadecimal format.